

PerfectPunch

User Guide

Boxing session analysis web app — setup, run, and use

1. What is PerfectPunch?

PerfectPunch is a web application that analyses boxing sensor recordings: an athlete uploads a single-arm or dual-arm ZIP, the app runs a trained Python model to detect punch events, and saves the result as a session that can be reviewed, edited, compared, filtered, favourited and exported as a PDF report.

2. Core features

- User signup and login, with per-user data isolation.
- Single-arm or dual-arm ZIP upload, with an optional session date override.
- Punch event analysis using a trained Python model (CNN/MLP-style classifier).
- Dashboard with per-user stats and a progress-over-time chart.
- Session detail page with arm-level breakdowns and a per-event log.
- Edit a saved session (title, notes) inline.
- Delete a saved session — removes both the database rows and the on-disk ZIPs.
- Favourite or unfavourite a saved session, from the History page or the detail page.
- History search and filter: text search over title/notes, favourites only, arm filter (left, right, both), date range.
- Side-by-side compare for any two saved sessions, with delta chips for events, uncertain count and clean rate.
- PDF export of any session report (browser-based print, no extra dependency).

3. System requirements and dependencies

- Node.js 18 or higher.
- Python 3.10 or higher.
- MySQL 8 or higher, running locally.
- Modern desktop browser (Chrome, Safari, Firefox, Edge) for the print-to-PDF feature.

Install the language-level dependencies:

```
# Node dependencies  
npm install
```

```
# Python dependencies (in a virtual environment)
```

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

4. Setup and run instructions

Follow these steps in order. They assume a fresh machine with Node, Python and MySQL already installed.

4.1 Clone and install

```
git clone https://github.com/HCosk/PerfectPunch.git
cd PerfectPunch

npm install

python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

4.2 Create the MySQL database and user

Open a MySQL shell as root and run:

```
CREATE DATABASE perfectpunch;
CREATE USER 'perfectpunch'@'localhost' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON perfectpunch.* TO 'perfectpunch'@'localhost';
FLUSH PRIVILEGES;
```

4.3 Import the schema

```
mysql -u perfectpunch -p perfectpunch < database/schema.sql
```

If "mysql: command not found" appears, use the full path of your installed MySQL client, for example `/usr/local/mysql-8.4.7-macos15-arm64/bin/mysql`.

4.4 Configure environment variables

Copy the template and edit it:

```
cp .env.example .env
```

At minimum, set `DB_PASSWORD` to the password you used in step 4.2. Leave `APP_BASE_PATH` empty for local runs.

4.5 Start the server

```
npm start
```

When you see 'PerfectPunch listening on `http://localhost:3000`', open that URL in a browser.

On boot, the server runs an idempotent migration that adds the `is_favorite` column to existing databases. No manual SQL is needed when upgrading an older database.

5. Environment variables

The full `.env.example` file looks like this:

```
PORT=3000
```

```
APP_BASE_PATH=  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_USER=perfectpunch  
DB_PASSWORD=change-me  
DB_NAME=perfectpunch  
SESSION_COOKIE_NAME=perfectpunch_session  
SESSION_COOKIE_SECURE=0  
SESSION_DAYS=30  
UPLOAD_PAYLOAD_LIMIT_MB=80  
PYTHON_BIN=python3
```

What each key does:

- PORT — local HTTP port for the app.
- APP_BASE_PATH — leave empty for local runs; set to the proxy prefix (for example /usr/123) for the Goldsmiths VM deploy.
- DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME — MySQL connection details.
- SESSION_COOKIE_NAME — the cookie used for login sessions.
- SESSION_COOKIE_SECURE — set to 1 to mark the cookie Secure (HTTPS-only).
- SESSION_DAYS — how many days a login session lasts.
- UPLOAD_PAYLOAD_LIMIT_MB — maximum upload payload size for a session ZIP, in megabytes.
- PYTHON_BIN — path to the Python interpreter used by the analyzer; defaults to python3.

6. Test credentials and sample inputs

No default user is shipped with the app — accounts are created at runtime by the user. To create a test account, open <http://localhost:3000/signup> and fill in any values. For example:

- Username: tester
- Email: tester@example.com
- Password: Password123!

Sample ZIPs: any ZIP that follows the project's internal CSV structure (the same format used by the training pipeline) can be uploaded. The Python CLI command `python3 app/cli.py analyze --zip-path <path>` uses the same code path as the web upload, so any ZIP that works on the CLI works in the UI.

A trained model is committed to `artifacts/current/`, so the app works out of the box without retraining.

6.5 Collecting a session ZIP

Recordings are captured with the Sensor Logger app by Choi Tsz Hei (see References, section 11) running on a smartwatch or smartphone, then exported as a ZIP and uploaded through the Record session form. To produce a usable ZIP:

- Install the Sensor Logger app on the phone or smartwatch you will wear during the round.
- In the app's sensor settings, enable Accelerometer, Gyroscope and Orientation. These three streams are required — the analyzer rejects ZIPs that are missing any of them.
- Start the recording just before the round, and stop it as soon as the round ends.
- Export the recording from Sensor Logger as a ZIP archive.
- For dual-arm sessions, repeat the steps above on a second device worn on the other arm — one ZIP for the left arm and one for the right.

Upload the ZIPs through the Record session form (see section 7.2).

7. How to use the app — click by click

7.1 Sign up and log in

1. Open <http://localhost:3000> in a browser. You are redirected to /login.
2. Click 'Create one' (or open /signup directly).
3. Fill in username, email, password, then click 'Create account'.
4. You are signed in automatically and redirected to the Dashboard.
5. To log out, click 'Log out' in the top-right of the page. To log back in, use the username (or email) and password from signup.

7.2 Record a session

Single-arm flow:

6. Click 'Record session' in the top navigation.
7. Type a session title (for example 'Tuesday sparring review').
8. Optionally write notes (what you were working on in this round).
9. Optionally pick a date and time under 'Optional session date override'. Leave both empty to use the timestamp inside the upload metadata.
10. Leave the mode set to 'Single arm'.
11. Pick the arm you are uploading (left or right).
12. Click the file input and choose your .zip file.
13. Click 'Analyse and save session'. The status line shows progress.
14. When analysis finishes, you are redirected to the new session's detail page.

Dual-arm flow:

15. On the same form, click 'Both arms'.
16. Upload one ZIP for the left arm and one for the right arm.
17. Click 'Analyse and save session'.

7.3 View the Dashboard

Click 'Dashboard' in the top navigation. You see five summary stat cards (Recorded sessions, Arm uploads, Detected events, Uncertain rate, Top punch), a 'Recent sessions' table with the

six most recent sessions, a 'Punch mix' bar chart, and a 'Progress over time' line chart of the most recent twelve sessions.

7.4 View a session

18. Click 'History' in the top navigation, or open the dashboard.
19. Click any session title in the table — the row link opens the session detail page.
20. On the detail page you see the title, notes, recorded date and total events at the top, an action row (Favourite, Edit, Export PDF, Compare with..., Delete), four stat cards, an overall punch breakdown, and one panel per uploaded arm with a metadata grid, a punch breakdown bar chart, and a per-event log table.
21. If a non-jab arm has jab labels detected, that arm's panel shows a warning message.

7.5 Edit a session

22. On a session detail page, click 'Edit'.
23. An inline form opens with the current title and notes.
24. Edit the title and/or notes.
25. Click 'Save changes'. The page reloads with the updated values.
26. If you want to discard the edit instead, click 'Cancel'. The form closes and nothing is saved.
27. If you clear the title and click 'Save changes', the server returns the message 'Title cannot be empty.' Use the back button and try again.

7.6 Favourite a session

From the History page:

28. Each row has a star at the start of the row (an empty star for not favourited).
29. Click the star to toggle. The page reloads briefly and the star fills in (gold).
30. Click again to unfavourite.

From a session detail page:

31. Click the 'Favourite' button in the action row.
32. The button label changes to 'Favourited' and the page eyebrow now shows 'Saved session · favourite'.
33. Click again to unfavourite.

7.7 Delete a session

34. On a session detail page, click 'Delete' (the red-tinted button on the right of the action row).
35. A browser confirmation dialog asks 'Delete this session and its uploads?'.
36. Click 'Cancel' to abort, or 'OK' to delete.
37. If you confirm, the database rows for the session are removed (the per-arm rows are removed by the foreign-key cascade), the matching storage/uploads/<uuid>/ folder on disk is removed, and you are redirected to History.

Deletion is immediate and cannot be undone.

7.8 Search and filter the History

Open History. Above the sessions table is a filter bar with five controls and a Reset button. Filtering is client-side, so changes apply instantly.

- Search: type any text. Rows whose title or notes do not contain that text are hidden.
- Arm: pick 'Left only', 'Right only' or 'Both arms' to narrow by arm composition.
- From / To: pick a start and end date to keep only sessions whose recorded date falls in that range.
- Favourites only: tick to show only favourited sessions.
- Reset: click to clear all filters at once.

If no rows match, the table area shows 'No sessions match these filters.'

7.9 Compare two sessions

From the History page:

38. Each row has a checkbox at the very start of the row (next to the favourite star).
39. Tick the checkbox on two different rows. A toolbar above the table updates: 'Pick 1 more session to compare.' becomes 'Two sessions picked — ready to compare.' once you have two.
40. Click 'Compare selected'. You are taken to `/sessions/compare?a=ID1&b=ID2`.
41. If you try to tick a third checkbox while two are already ticked, the third one is automatically un-ticked. The cap is two.

From a session detail page:

42. Click 'Compare with...' in the action row.
43. A browser prompt asks for the other session id. Find an id by visiting another session — it is the number at the end of the URL, for example `/sessions/42` means id 42.
44. Type the id and click OK. You are taken to the comparison page.
45. If the id does not exist for your account, you see 'Cannot compare sessions'.

On the compare page you see:

- A title 'Session A title vs Session B title'.
- Four stat cards with delta chips: Events Δ , Uncertain Δ , Clean rate Δ , Upload modes. Positive deltas are green, negative deltas are red, zero is neutral.
- Two side-by-side cards with each session's punch breakdown bar chart.
- Two side-by-side columns with each session's arm panels.
- A 'Back to history' link in the top-right.

7.10 Export a session as PDF

46. Open a session detail page.

47. Click 'Export PDF' in the action row. A new tab opens at /sessions/<id>/print with a clean, print-only layout (no top navigation, no buttons).
48. Click 'Save as PDF' in the top-right of that page (or press Cmd+P / Ctrl+P).
49. In your browser's print dialog, set the destination to 'Save as PDF' and click Save.
50. The downloaded PDF contains the title, notes, recorded date, the four stat cards, the overall punch breakdown, and one panel per arm with metadata, a breakdown chart and the per-event log.

PDF export uses the browser's print engine on purpose — this avoids a heavy native dependency such as puppeteer or wkhtmltopdf.

8. Quick smoke test (about 2 minutes)

Run through these in order to exercise every feature once. Make sure you have at least three saved sessions before you start.

51. Log in.
52. Open History. Click the empty star on one row to favourite it; the star turns gold.
53. Click that session's title. On the detail page, click 'Edit', change the title, then click 'Save changes'. The page reloads with the new title.
54. Click 'Export PDF'. The print page opens in a new tab. Close it.
55. Go back to History. Type 'test' in the Search field; the table filters live.
56. Tick the 'Favourites only' box; the list shows only favourited rows. Untick.
57. Click 'Reset' to clear all filters.
58. Tick the checkboxes on two different rows; the compare toolbar enables the 'Compare selected' button. Click it. The compare page opens with delta chips. Click 'Back to history'.
59. Open one of the test sessions you no longer need, click 'Delete', then click 'OK' in the confirmation dialog. The row disappears from History.
60. Click 'Log out' in the top-right to end the smoke test.

9. Known limitations and what is not implemented yet

- PDF export uses the browser's 'Save as PDF' dialog rather than a server-rendered PDF. This avoids a heavy native dependency but means the layout depends on the user's browser print settings.
- Compare is limited to exactly two sessions. Three-way and N-way comparisons are not supported.
- The 'Compare with...' button on the session detail page uses a browser prompt for the other session id, not a searchable picker. The History page checkbox flow is the recommended path.
- History filtering is client-side over the full set of the user's sessions. With very large session counts (thousands) this would need pagination and server-side filters; the current build is sized for a single athlete's history.
- There is no bulk export and no bulk delete. Each operation is per session.

- Deletion has no undo. Both the database rows and the on-disk ZIPs are removed immediately.
- There are no e-mail flows: no password reset, no e-mail verification.
- There is no admin UI for retraining the model. Training is terminal-only via `python3 app/cli.py train`, and a fresh model snapshot is committed to `artifacts/current/`.
- There is no automated JavaScript test suite. Python tests cover the model pipeline (`tests/test_api.py`, `tests/test_pipeline.py`) and `npm run check` only does syntax checks on the Node server.

10. Troubleshooting

'mysql: command not found'

Your shell cannot find the MySQL client. Use the full path, for example `/usr/local/mysql-8.4.7-macos15-arm64/bin/mysql`, or add the bin directory to `PATH`.

'Access denied for user perfectpunch@localhost'

The `DB_PASSWORD` in `.env` does not match the one you set when creating the MySQL user. Edit `.env` and restart the server.

Port 3000 is already in use

Find the process listening on the port and stop it:

```
lsof -nP -iTCP:3000 -sTCP:LISTEN
# Note the PID column in the output, then:
kill <PID>
```

Then run `npm start` again.

Model artefacts not ready

If the upload page shows 'Model artefacts are not ready yet.', train the model from the terminal once:

```
source .venv/bin/activate
python3 app/cli.py train
python3 app/cli.py info
```

After training, restart the server. Note that the project also commits a working `artifacts/current/` folder, so this should rarely be needed.

Proxy page not loading after restart (Goldsmiths VM)

Wait up to about a minute. The proxy can take a moment to pick up the restarted process.

'Title cannot be empty.' after editing

Title is required. Click back, type a title, and save again.

11. References

- Choi, T. H. Sensor Logger [Mobile application]. Used to capture accelerometer, gyroscope and orientation streams during boxing rounds; recordings are exported as ZIP archives and uploaded to PerfectPunch.